Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 15 · Part D: Digital Signal Processing Level 3

Track: Recursive Filters, Bilinear Transform, Numerical Stability, Fixed-Point Biquads, Real-Time Detection

Prerequisites: Problems 06, 08, 14 · **Estimated effort:** 12–16 hours

IIR Filters and Real-Time Implementation: Few Coefficients, Many Ways to Go Wrong

Turn analog designs into recursive digital filters with the bilinear transform, compare their cost with FIR designs, discover why a high-order filter written as one polynomial blows up while the same filter as cascaded sections works, implement a Q15 fixed-point biquad, and build a real-time apogee detector for a model rocket from a noisy barometer.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/15_iir_filters_and_real_time_implementation/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random

seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

Recursive (IIR) filters achieve a given selectivity with an order ten times lower than FIR filters, which is why they dominate on small processors and in real-time loops. The price is nonlinear phase, a possibility of instability, and sensitivity to coefficient rounding: a narrow filter has poles crowded near $z = 1$, where tiny changes of coefficients move the poles across the unit circle.

For a flight computer the crucial additional constraint is *causality*. A zero-phase filter (forward-backward filtering) is excellent for post-flight analysis but cannot be used in flight because it needs the future. A real-time filter has delay, and that delay turns directly into a late detection. In this problem you design, implement and stress IIR filters and use them in a safety-relevant application: detecting the apogee of a rocket to fire the parachute ejection charge.

Theoretical Framework

1. Bilinear Transform and Prewarping

Substituting $s = \frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}$ into an analog prototype $H_a(s)$ maps the left half-plane into the unit disc and the whole $j\omega$ axis onto the unit circle, with frequency warping

$$\Omega = \frac{2}{T}\tan\frac{\omega T}{2}, \qquad \omega T = 2\arctan\frac{\Omega T}{2}. \tag{1}$$

To place a digital critical frequency ω_c exactly, design the analog prototype at the prewarped $\Omega_c = \frac{2}{T}\tan(\omega_c T/2)$. In SciPy the critical frequency `Wn` is normalised to the Nyquist frequency $f_s/2$.

2. Families and Zero-Phase Filtering

Butterworth, Chebyshev I/II and elliptic prototypes carry over unchanged; the orders needed are the analog orders of problem 08. Forward–backward filtering ($y = \mathrm{filtfilt}(x)$) gives zero phase but squares the magnitude, so the -3 dB point becomes -6 dB and the effective order doubles; it requires the entire record.

3. Structures and Numerical Sensitivity

Direct form I/II implements $\sum b_k z^{-k} / \sum a_k z^{-k}$ with the full polynomial. For high order or a narrow band the poles are very sensitive to the coefficients. Factoring into second-order sections (SOS, biquads), each realised separately and cascaded, keeps each pole pair sensitive only to its own few coefficients; ordering and gain scaling of the sections control internal overflow and noise.

4. Fixed-Point Implementation

In Q15 arithmetic a coefficient $c \in [-1, 1)$ is stored as $\text{round}(c \cdot 2^{15})$; products need a wider accumulator (32-bit) and results are shifted back. Overflow must saturate rather than wrap. Coefficients with magnitude above 1 (such as $a_1 \approx -2$ in a biquad) are handled by scaling or by splitting. Quantised poles move; with zero input a recursive filter may sustain a small *limit cycle*.

5. Exponential Smoothing and Complementary Filters

$y[n] = \alpha y[n-1] + (1-\alpha)x[n]$ has its -3 dB frequency at $\omega_c = \arccos\left[1 - (1-\alpha)^2/(2\alpha)\right]$ rad/sample. A complementary filter combines a low-passed absolute sensor (barometer, accelerometer) with a high-passed integrating sensor so that the two cross over smoothly.

Data and Simulation Specification

Specification B	$f_s = 1$ kHz, passband to 100 Hz (1 dB), stopband from 200 Hz (50 dB); the FIR comparison filter is the Specification A filter of problem 14 with relaxed attenuation.
Narrowband stress test	8th-order Butterworth low-pass with cutoff 1 Hz at $f_s = 1$ kHz ($W_n = 0.002$).
Fixed-point	Q15 biquad: Butterworth 2nd-order low-pass, cutoff $f_s/20$; input amplitude 0.5 full scale plus a zero-input test.
Barometer	`sp_common.synth.barometer_rocket(fs=20, duration=60)`: pressure in Pa with noise, drift and the ejection-charge pressure spike; `sp_common.units.baro_altitude_m` converts to altitude. 200 noise seeds for statistics; apogee near 258 m at about 7.6 s.
Real data (optional)	Phyphox barometer on a phone carried up and down a stairwell or in a lift, sampled at the maximum rate.

Your Task

1. **Bilinear transform by hand (Stage 2).** Derive the digital biquad coefficients of a 2nd-order Butterworth low-pass by substituting the bilinear map into the analog prototype (polynomial algebra in NumPy, no `scipy.signal.bilinear`). Verify against `scipy.signal.butter` and check that the -3 dB point is at the prewarped frequency.
2. **Warping.** Compare the magnitude response of an uncorrected bilinear design with the analog original for cutoff frequencies $f_c/f_s = 0.01, 0.1, 0.3, 0.45$; plot the frequency error and show that prewarping fixes the cutoff but not the shape far from it.
3. **IIR versus FIR cost.** Meet Specification B with Butterworth, Chebyshev I and elliptic filters. Tabulate order, number of coefficients, multiply–accumulates per sample and group-delay variation, and compare with the FIR filter that meets the same specification.
4. **Numerical stability.** Design the narrowband 8th-order Butterworth as a single transfer function (`b, a`) and as SOS. Compare their responses in float64 and float32, plot the poles, and

show the instability or gross error of the direct form. Perturb the coefficients randomly at a relative level of 10^{-8} and measure the pole movement for both structures.

5. **Fixed-point biquad.** Implement a Q15 direct-form biquad with a 32-bit accumulator and saturation; compare its output with float64 over a sine input and compute the noise floor in dB below full scale. Observe the zero-input limit cycle and quantify the pole displacement caused by coefficient rounding.
6. **Zero-phase versus real-time.** Filter the barometer data with `lfilter` and with `filtfilt`; plot the delay between the filtered and true altitude and between their apogee times. Design a causal apogee detector: low-pass the altitude, estimate velocity, trigger after N consecutive negative-velocity samples and arm only after burnout. Over 200 noise seeds report detection delay, altitude error at trigger, and the false-trigger rate caused by noise and by the ejection spike; tune the filter cutoff and N to balance delay against false triggers.
7. **Exponential and complementary filtering.** Verify the α–ω_c relation for $\alpha = 0.9$. Build a complementary filter that fuses the barometer altitude with the double-integrated accelerometer (simulated from the true acceleration plus bias and noise) and show the improvement in lag and noise.

Self-check targets

For $\alpha = 0.9$ the exponential smoother has $\omega_c = 0.1055$ rad/sample. A 2nd-order Butterworth with the correct prewarp has exactly -3.01 dB at the design frequency; `filtfilt` gives -6.02 dB there. The direct-form narrowband 8th-order filter is unusable in float32 while the SOS version agrees with float64 to better than 10^{-5}. The IIR designs meeting Specification B have order 4–6 against an FIR length of some tens of taps.

Aerospace Application Link

A model-rocket flight computer must eject the parachute within a short window around apogee, but not during boost when transonic pressure disturbances can mimic a descent. Using Task 6, report the detection delay and the false-trigger rate of your best design, the altitude lost between apogee and ejection ($\approx \frac{1}{2} g t^2$ for a delay t), and the arming logic (minimum time after launch, minimum altitude, burnout detection) that keeps the false-trigger rate below one in 10^4 flights. State whether a backup timer is justified.

Deliverables

D1 **Bilinear design:** derivation, hand-coded coefficients and the warping study.

D2 **Family comparison:** order and cost table versus the FIR design.

D3 **Numerical stability:** float32 versus float64, direct versus SOS, pole-sensitivity measurements.

D4 **Fixed-point biquad:** Q15 implementation, noise floor, limit-cycle and coefficient-rounding study.

D5 **Apogee detector:** the causal design, statistics over 200 seeds and the arming logic.

D6 **Memo (300 words):** "Detecting apogee safely: delay versus false triggers." Cite at least two external sources.

Bonus (a) Add a Kalman filter for altitude and velocity (problem 20) and compare delay with your filter. (b) Implement the detector in C or MicroPython and run it on a microcontroller with

recorded data. (c) Compare Direct-Form I, II and transposed II in fixed point for overflow behaviour.

Learning Resources

IIR Design and Implementation

- Oppenheim, A.V., Schafer, R.W. *Discrete-Time Signal Processing*, Ch. 6 (structures for discrete-time systems), Ch. 7.1–7.3 (IIR design) and Ch. 6.8–6.9 (finite word length effects).
- Proakis, J.G., Manolakis, D.G. *Digital Signal Processing*, Ch. 8–9.
- Orfanidis, S.J. *Introduction to Signal Processing* (free online), chapters on IIR filter design and structures.

Flight Systems

- Stine, G.H. *Handbook of Model Rocketry* (barometric recovery and ejection charges).
- Madgwick, S.O.H., Harrison, A.J.L., Vaidyanathan, R. (2011). "Estimation of IMU and MARG orientation using a gradient descent algorithm." IEEE Int. Conf. Rehabilitation Robotics (an example of complementary-style fusion).

Tools

- SciPy `signal.butter`, `signal.sosfilt`, `signal.filtfilt`, `signal.bilinear`: https://docs.scipy.org/doc/scipy/reference/signal.html

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.